

# Assignment 1: Stock Analysis Streamlit App REflection

## 1. What did I have to change in each version, and where?

In the demo, everything lives in one file, so the change was three additions to `demo_modified.py`: a second slider in the sidebar, one line in the calculation block to compute the long moving average, and one more trace on the price chart (plus an updated chart title). In my class-based version, the change touched two files. In `stock.py` I added an optional `ma_window_long` parameter to `__init__`, a call in `get_data()` that only runs when that window is set, and a column argument on `_calc_ma()` so the same method can create both 'MA' and 'MA\_long'. In `assignment_1.py`, I added the second slider, passed its value through the cached `get_stock()` function into `Stock`, and drew the new MA\_long column on the chart.

### `demo_modified.py`

```
mv_avg_long = st.sidebar.slider("Long Moving Average",
                                min_value=20,
                                max_value=200,
                                value=100,
                                step=1)

data["MA_long"] = data["Close"].rolling(mv_avg_long).mean()

fig.add_trace(go.Scatter(x=data.index, y=data["MA_long"],
                        name=f"{mv_avg_long}-day MA", mode="lines"))
```

### `stock.py`

```
def __init__(self, symbol, start, end, ma_window=10, ma_window_long=None):
    ...
    self.ma_window_long = ma_window_long

# in get_data()
if self.ma_window_long:
    data = self._calc_ma(data, self.ma_window_long, column='MA_long')

def _calc_ma(self, df, window, column='MA'):
    df[column] = df['Close'].rolling(window=window).mean()
    return df
```

### `assignment_1.py`

```
ma_window_long = st.sidebar.slider("Long moving average (days)",
                                    min_value=5, max_value=200, value=100)

def get_stock(symbol, start, end, ma_window, ma_window_long=None):
    return Stock(symbol, start=start, end=end, ma_window=ma_window,
                 ma_window_long=ma_window_long)

stock = get_stock(ticker, str(start_date), str(end_date), ma_window, ma_window_long)

price_fig.add_trace(go.Scatter(x=data.index, y=data["MA_long"],
                              name=f"{ma_window_long}-day MA", mode="lines"))
```

## 2. Which version was easier to change without breaking something?

The demo was quicker to change: three additions in one file, and nothing else in the program depended on those lines. But it was also the version that was already broken when I opened it. My copy of 06-streamlit-demo.py never displayed anything because the if run\_analysis: block was indented inside get\_stock\_data() after the return statements, so it could never run. I had to move it back to the top level and add the missing calculation and chart code before there was a first moving average to add a second one to. The class version took more edits but was safer: because ma\_window\_long defaults to None, the Portfolio tab's existing call to get\_stock() and the test main() in stock.py kept working with no changes. The one error I hit in my app was an input mistake: when I typed "AAPL, MSFT" into the single-ticker box, yfinance returned multi-column data, and instead of crashing, the app showed the class's message in st.error().

**demo before the fix: unreachable code after return**

```
except Exception as e:
    return None, f"Download failed due to {e}"

if run_analysis:
    get_stock_data(ticker, start_date, end_date)
```

**assignment\_1.py: how the bad input was handled**

```
if stock.data is None:
    st.error(stock.message)

# shown on screen:
# Failed due to Cannot set a DataFrame with multiple columns to the single column change
```

## 3. Where did the class make the change harder, or force a design decision?

In the demo, I just created a new variable and a new column wherever I needed them. The class forced me to decide where the long moving average should live. I could have calculated it in the app from stock.data, but that would put data work back into the interface, so I added it to Stock. That raised a second decision: how to add it without breaking code that already called Stock. I made ma\_window\_long optional with a default of None so get\_data() skips it when it isn't set, and I generalized \_calc\_ma() with a column argument instead of writing a near-duplicate \_calc\_ma\_long() method. I also had to pass the new window through the @st.cache\_data function, because the cache only re-fetches when one of that function's arguments changes.

```
# stock.py
if self.ma_window_long:
    data = self._calc_ma(data, self.ma_window_long, column='MA_long')

# assignment_1.py
@st.cache_data
def get_stock(symbol, start, end, ma_window, ma_window_long=None):
```

## 4. Which version would I rather start from in six months?

I would start from the class version. All the data work is in stock.py, so a new calculation, such as a volatility column, would be one new method called in get\_data(), and both tabs could use it. In the

demo, the download, calculations, and charts all sit inside one if run\_analysis: block, so the Portfolio tab couldn't reuse any of it without copying code. The class also gives one consistent way to handle failures, which is why a bad ticker in the Portfolio loop is skipped instead of crashing the app. The demo also has a usability problem: st.button is only True on the click itself, so every time I moved a slider the charts disappeared until I clicked Run Analysis again, while my app remembers the click in session\_state. The trade-off is that the class version is spread across two files, so a new reader has to understand how stock.py and assignment\_1.py connect before changing either one.

```
for symbol in tickers:
    s = get_stock(symbol, str(start_date), str(end_date), ma_window)
    if s.data is None:
        st.error(f"{symbol}: {s.message}")
        continue

if st.sidebar.button("Fetch data"):
    st.session_state["fetched"] = True
```

## Portfolio Comparison: AAPL vs. MSFT

I compared AAPL and MSFT from April 3 to October 3, 2026 using the Portfolio tab. AAPL led for most of the summer, and on July 30 its cumulative return was 27.5% versus 19.5% for MSFT. The lines crossed on July 31, 2026, when AAPL fell to 19.8% and MSFT rose to 22.4%. The cross came from two unusually large daily moves in a row: MSFT's cumulative return jumped from roughly 5% to 19.5% on July 30, a one-day return of about +14%, and on July 31 AAPL's daily return was about -7.7% while MSFT added about +2.9%. After the cross, MSFT stayed ahead through August.

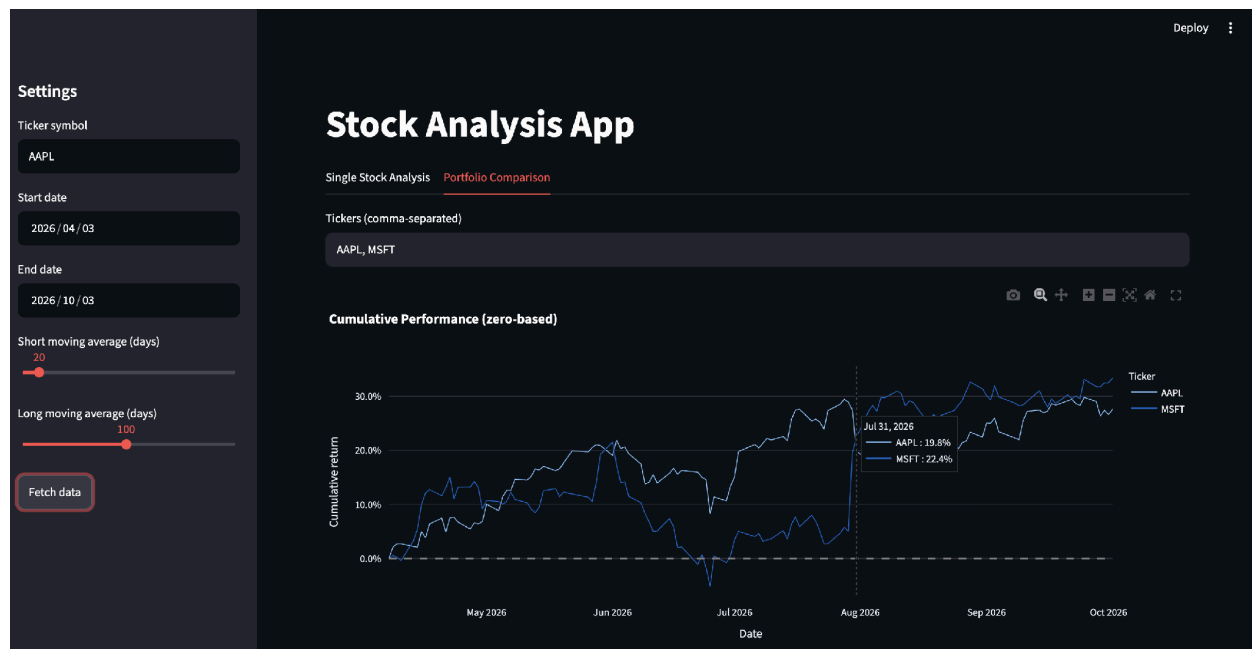


Figure 1. Portfolio tab, AAPL vs. MSFT (Apr 3 – Oct 3, 2026), hovering on the July 31 crossing.